

Day 3: Build Your First Multi-Tool Customer-Service Agent with Gemini Function Calling

Text from Claude: I'll generate this `guide.md` for you directly. The `guide.md` is complete. A few things worth flagging so you can quality-check it before handing it to interns:

Deliberate decisions I made:

- The `agent.py` uses `google.generativeai` (the `google-generativeai` pip package), not the newer `google-genai` SDK. The two have different import paths and schema classes. The `google-generativeai` package is more stable and better documented for this exact use case as of mid-2024. If you want the newer `google-genai` SDK, the schema construction syntax changes significantly — worth being consistent across your course materials.
- The manual loop sends the `function_response` as a raw `Content` protobuf object. This is the correct pattern for the `google-generativeai` SDK but interns will likely trip on it. The troubleshooting section calls this out explicitly.
- I used `gemini-1.5-flash` as the default model. If your course uses `gemini-2.0-flash` or another version, update the `.env.example` default and test the schema compatibility — newer models can be more aggressive about automatic function calling even when you haven't enabled it.

One gap to be aware of: The `agent.py` schema uses `genai.protos.Tool` and `genai.protos.FunctionDeclaration`. Some older versions of the `google-generativeai` package use slightly different import paths. If interns hit import errors on those, the fix is pinning to `google-generativeai>=0.7.0` which is already in `requirements.txt`.

What You're Building and Why

Today you'll build a customer-service agent that can look up orders, check shipping, explain refund policies, and escalate angry customers — all by giving Gemini a set of Python tools and letting it decide which ones to call.

This is called **function calling** (also called tool use). It's the core pattern behind almost every real-world AI agent. You are not teaching Gemini new facts — you are giving it *actions* it can take, and it figures out when to take them.

The cycle looks like this:

```
User message
  ↓
Gemini reads the message and your tool definitions
  ↓
Gemini says: "I need to call get_order(order_id='1003')"
  ↓
Your Python code runs get_order('1003') and gets the result
  ↓
```

You send that result back to Gemini

↓

Gemini reads the result and either calls another tool or gives a final answer

↓

Repeat until Gemini gives a final natural-language answer

You will build this loop manually so you understand every step. Once you've done it by hand, you'll understand why automatic function calling exists — and when to trust it.

Final Folder Structure

```
fc_agent/  
  agent.py  
  tools.py  
  fake_orders.json  
  test_queries.md  
  test_results.md  
  escalation_logic.md  
  learnings.md  
  requirements.txt  
  .env.example
```

Step 1: Create the Project Folder and Files

macOS / Linux

```
mkdir fc_agent  
cd fc_agent  
touch agent.py tools.py fake_orders.json test_queries.md test_results.md  
escalation_logic.md learnings.md requirements.txt .env.example
```

Windows PowerShell

```
mkdir fc_agent  
cd fc_agent  
New-Item agent.py, tools.py, fake_orders.json, test_queries.md,  
test_results.md, escalation_logic.md, learnings.md, requirements.txt,  
.env.example -ItemType File
```

Step 2: DeepSeek Copy-Paste Prompts

For each file below, you'll find a prompt you can paste into DeepSeek Chat to generate the file contents, followed immediately by the complete recommended file contents you should use or compare against.

fake_orders.json

DeepSeek Prompt:

Generate a fake_orders.json file for a Python customer-service agent project.

Requirements:

- Create a JSON object where each key is an order_id string (e.g. "1001", "1002", ..., "1022").
- Create at least 22 orders.
- Each order must include these fields:
 - order_id, customer_name, email, category, item, price (float), order_status, shipping_status, tracking_number, estimated_delivery, delivered_date (null if not delivered), return_window_days (int), refund_status, notes.
- order_status values: delivered, in_transit, returned, disputed, cancelled, processing
- shipping_status values: delivered, in_transit, not_shipped, returned, cancelled
- categories: electronics, clothing, home, beauty, books, toys, groceries
- refund_status values: not_requested, approved, denied, pending, processed
- Make the data realistic and varied. Include some orders with notes about issues (e.g. "Item arrived damaged", "Customer reported missing package").
- Output only the raw JSON, no explanation.

Complete File Contents:

```
{
  "1001": {
    "order_id": "1001",
    "customer_name": "Alice Johnson",
    "email": "alice.j@email.com",
    "category": "electronics",
    "item": "Wireless Noise-Cancelling Headphones",
    "price": 149.99,
    "order_status": "delivered",
    "shipping_status": "delivered",
    "tracking_number": "TRK-AA1001",
    "estimated_delivery": "2024-05-10",
    "delivered_date": "2024-05-09",
    "return_window_days": 30,
    "refund_status": "not_requested",
    "notes": "Delivered one day early."
  },
  "1002": {
```

```
    "order_id": "1002",
    "customer_name": "Brian Patel",
    "email": "bpatel@email.com",
    "category": "clothing",
    "item": "Men's Running Jacket",
    "price": 79.95,
    "order_status": "in_transit",
    "shipping_status": "in_transit",
    "tracking_number": "TRK-BP1002",
    "estimated_delivery": "2024-06-15",
    "delivered_date": null,
    "return_window_days": 60,
    "refund_status": "not_requested",
    "notes": "Package last scanned in Chicago distribution center."
  },
  "1003": {
    "order_id": "1003",
    "customer_name": "Carla Nguyen",
    "email": "carla.n@email.com",
    "category": "home",
    "item": "Stainless Steel Air Fryer",
    "price": 89.00,
    "order_status": "delivered",
    "shipping_status": "delivered",
    "tracking_number": "TRK-CN1003",
    "estimated_delivery": "2024-05-20",
    "delivered_date": "2024-05-21",
    "return_window_days": 30,
    "refund_status": "not_requested",
    "notes": ""
  },
  "1004": {
    "order_id": "1004",
    "customer_name": "David Kim",
    "email": "dkim@email.com",
    "category": "electronics",
    "item": "USB-C Hub 7-in-1",
    "price": 44.99,
    "order_status": "returned",
    "shipping_status": "returned",
    "tracking_number": "TRK-DK1004",
    "estimated_delivery": "2024-04-28",
    "delivered_date": "2024-04-27",
    "return_window_days": 30,
    "refund_status": "processed",
    "notes": "Customer returned due to compatibility issues. Refund of $44.99 issued."
  },
  "1005": {
    "order_id": "1005",
    "customer_name": "Emma Rivera",
    "email": "emma.r@email.com",
    "category": "beauty",
    "item": "Vitamin C Serum 30ml",
```

```
    "price": 32.50,
    "order_status": "disputed",
    "shipping_status": "delivered",
    "tracking_number": "TRK-ER1005",
    "estimated_delivery": "2024-05-05",
    "delivered_date": "2024-05-06",
    "return_window_days": 14,
    "refund_status": "pending",
    "notes": "Customer claims item arrived damaged. Dispute opened 2024-05-08. Awaiting photo evidence."
  },
  "1006": {
    "order_id": "1006",
    "customer_name": "Frank Osei",
    "email": "fosei@email.com",
    "category": "books",
    "item": "Python for Data Analysis, 3rd Edition",
    "price": 54.99,
    "order_status": "delivered",
    "shipping_status": "delivered",
    "tracking_number": "TRK-F01006",
    "estimated_delivery": "2024-05-12",
    "delivered_date": "2024-05-11",
    "return_window_days": 30,
    "refund_status": "not_requested",
    "notes": ""
  },
  "1007": {
    "order_id": "1007",
    "customer_name": "Grace Tanaka",
    "email": "grace.t@email.com",
    "category": "clothing",
    "item": "Women's Yoga Pants",
    "price": 49.00,
    "order_status": "delivered",
    "shipping_status": "delivered",
    "tracking_number": "TRK-GT1007",
    "estimated_delivery": "2024-05-18",
    "delivered_date": "2024-05-17",
    "return_window_days": 60,
    "refund_status": "not_requested",
    "notes": "Customer has asked about sizing exchange."
  },
  "1008": {
    "order_id": "1008",
    "customer_name": "Hassan Ali",
    "email": "h.ali@email.com",
    "category": "toys",
    "item": "LEGO Technic Set 42-piece",
    "price": 69.99,
    "order_status": "cancelled",
    "shipping_status": "cancelled",
    "tracking_number": null,
    "estimated_delivery": null,
```

```
"delivered_date": null,
"return_window_days": 30,
"refund_status": "processed",
"notes": "Order cancelled by customer before shipping. Full refund of
$69.99 issued."
},
"1009": {
  "order_id": "1009",
  "customer_name": "Isabelle Chen",
  "email": "ichen@email.com",
  "category": "groceries",
  "item": "Organic Oat Milk 6-Pack",
  "price": 18.00,
  "order_status": "delivered",
  "shipping_status": "delivered",
  "tracking_number": "TRK-IC1009",
  "estimated_delivery": "2024-05-22",
  "delivered_date": "2024-05-22",
  "return_window_days": 7,
  "refund_status": "not_requested",
  "notes": ""
},
"1010": {
  "order_id": "1010",
  "customer_name": "James Okonkwo",
  "email": "jokonkwo@email.com",
  "category": "electronics",
  "item": "Smart Home Security Camera",
  "price": 129.00,
  "order_status": "disputed",
  "shipping_status": "delivered",
  "tracking_number": "TRK-J01010",
  "estimated_delivery": "2024-05-14",
  "delivered_date": "2024-05-15",
  "return_window_days": 30,
  "refund_status": "pending",
  "notes": "Customer reports camera not connecting to Wi-Fi. Second
dispute this month. High escalation risk."
},
"1011": {
  "order_id": "1011",
  "customer_name": "Karen Walsh",
  "email": "kwalsh@email.com",
  "category": "home",
  "item": "Bamboo Cutting Board Set",
  "price": 34.99,
  "order_status": "processing",
  "shipping_status": "not_shipped",
  "tracking_number": null,
  "estimated_delivery": "2024-06-20",
  "delivered_date": null,
  "return_window_days": 30,
  "refund_status": "not_requested",
  "notes": "Order placed 2024-06-14. Still in warehouse processing
```

```
queue."
},
"1012": {
  "order_id": "1012",
  "customer_name": "Liam Torres",
  "email": "ltorres@email.com",
  "category": "clothing",
  "item": "Denim Jacket Classic Fit",
  "price": 95.00,
  "order_status": "in_transit",
  "shipping_status": "in_transit",
  "tracking_number": "TRK-LT1012",
  "estimated_delivery": "2024-06-18",
  "delivered_date": null,
  "return_window_days": 60,
  "refund_status": "not_requested",
  "notes": "Delayed by 3 days due to carrier backlog."
},
"1013": {
  "order_id": "1013",
  "customer_name": "Maya Gupta",
  "email": "maya.g@email.com",
  "category": "beauty",
  "item": "Retinol Night Cream",
  "price": 58.00,
  "order_status": "returned",
  "shipping_status": "returned",
  "tracking_number": "TRK-MG1013",
  "estimated_delivery": "2024-04-30",
  "delivered_date": "2024-04-30",
  "return_window_days": 14,
  "refund_status": "approved",
  "notes": "Customer allergic reaction. Return approved outside standard
window as exception."
},
"1014": {
  "order_id": "1014",
  "customer_name": "Nathan Brown",
  "email": "nbrown@email.com",
  "category": "books",
  "item": "The Art of Statistics",
  "price": 22.00,
  "order_status": "delivered",
  "shipping_status": "delivered",
  "tracking_number": "TRK-NB1014",
  "estimated_delivery": "2024-05-25",
  "delivered_date": "2024-05-24",
  "return_window_days": 30,
  "refund_status": "not_requested",
  "notes": ""
},
"1015": {
  "order_id": "1015",
  "customer_name": "Olivia Scott",
```

```
"email": "oscott@email.com",
"category": "toys",
"item": "Remote Control Monster Truck",
"price": 59.99,
"order_status": "delivered",
"shipping_status": "delivered",
"tracking_number": "TRK-0S1015",
"estimated_delivery": "2024-05-30",
"delivered_date": "2024-05-29",
"return_window_days": 30,
"refund_status": "not_requested",
"notes": "Gift order. Customer requested no invoice in package."
},
"1016": {
  "order_id": "1016",
  "customer_name": "Peter Muller",
  "email": "pmuller@email.com",
  "category": "electronics",
  "item": "Mechanical Keyboard TKL",
  "price": 109.00,
  "order_status": "disputed",
  "shipping_status": "delivered",
  "tracking_number": "TRK-PM1016",
  "estimated_delivery": "2024-06-01",
  "delivered_date": "2024-06-02",
  "return_window_days": 30,
  "refund_status": "denied",
  "notes": "Customer claims missing keycaps. Refund denied after
investigation found no manufacturing defect. Customer very frustrated."
},
"1017": {
  "order_id": "1017",
  "customer_name": "Quinn Reyes",
  "email": "qreyes@email.com",
  "category": "groceries",
  "item": "Cold Pressed Juice Bundle",
  "price": 27.50,
  "order_status": "cancelled",
  "shipping_status": "cancelled",
  "tracking_number": null,
  "estimated_delivery": null,
  "delivered_date": null,
  "return_window_days": 7,
  "refund_status": "processed",
  "notes": "Cancelled due to stock unavailability. Refund issued
automatically."
},
"1018": {
  "order_id": "1018",
  "customer_name": "Rachel Lee",
  "email": "rlee@email.com",
  "category": "home",
  "item": "Electric Standing Desk Converter",
  "price": 219.00,
```



```
"order_status": "in_transit",
"shipping_status": "in_transit",
"tracking_number": "TRK-RL1018",
"estimated_delivery": "2024-06-19",
"delivered_date": null,
"return_window_days": 30,
"refund_status": "not_requested",
"notes": "Large item – freight delivery scheduled."
},
"1019": {
  "order_id": "1019",
  "customer_name": "Samuel Obi",
  "email": "sobi@email.com",
  "category": "clothing",
  "item": "Merino Wool Sweater",
  "price": 112.00,
  "order_status": "delivered",
  "shipping_status": "delivered",
  "tracking_number": "TRK-S01019",
  "estimated_delivery": "2024-06-05",
  "delivered_date": "2024-06-04",
  "return_window_days": 60,
  "refund_status": "not_requested",
  "notes": ""
},
"1020": {
  "order_id": "1020",
  "customer_name": "Tina Vasquez",
  "email": "tvasquez@email.com",
  "category": "beauty",
  "item": "Lip Gloss Collection Set",
  "price": 19.99,
  "order_status": "processing",
  "shipping_status": "not_shipped",
  "tracking_number": null,
  "estimated_delivery": "2024-06-21",
  "delivered_date": null,
  "return_window_days": 14,
  "refund_status": "not_requested",
  "notes": "New order. Processing normally."
},
"1021": {
  "order_id": "1021",
  "customer_name": "Uma Patel",
  "email": "upatel@email.com",
  "category": "electronics",
  "item": "Portable Bluetooth Speaker",
  "price": 74.99,
  "order_status": "returned",
  "shipping_status": "returned",
  "tracking_number": "TRK-UP1021",
  "estimated_delivery": "2024-05-16",
  "delivered_date": "2024-05-15",
  "return_window_days": 30,
```

```

    "refund_status": "pending",
    "notes": "Speaker stopped working after 2 days. Return in progress,
refund pending warehouse inspection."
  },
  "1022": {
    "order_id": "1022",
    "customer_name": "Victor Huang",
    "email": "vhuang@email.com",
    "category": "toys",
    "item": "Wooden Puzzle Set 500-piece",
    "price": 28.00,
    "order_status": "disputed",
    "shipping_status": "delivered",
    "tracking_number": "TRK-VH1022",
    "estimated_delivery": "2024-06-08",
    "delivered_date": "2024-06-09",
    "return_window_days": 30,
    "refund_status": "pending",
    "notes": "Customer says puzzle arrived with 47 missing pieces. Photos
submitted. Under review."
  }
}

```

tools.py

DeepSeek Prompt:

Write a Python file called tools.py for a customer-service agent project. It must implement exactly 4 functions:

1. `get_order(order_id: str) -> dict`
 - Load order data from `fake_orders.json` (located in the same directory).
 - If the `order_id` exists, return the order dict.
 - If not found, return `{"error": True, "message": f"Order {order_id} not found."}`.
2. `get_shipping(order_id: str) -> dict`
 - Load from `fake_orders.json`.
 - Return a dict with: `order_id`, `shipping_status`, `tracking_number`, `estimated_delivery`, `delivered_date`.
 - If not found, return `{"error": True, "message": f"No shipping info for order {order_id}."}`.
3. `check_refund_policy(category: str) -> dict`
 - Do NOT load from JSON. Use a hardcoded dict inside this function.
 - Categories and policies: electronics (30 days, no refund if opened unless defective), clothing (60 days, free returns, exchange available), home (30 days, must be unused), beauty (14 days, unopened only, exceptions for allergic reactions), books (30 days, no refund on digital), toys (30 days, must be in original packaging), groceries (7 days, perishables non-

```
refundable, exceptions for damaged goods).
    - Return: {"category": ..., "return_window_days": ..., "conditions":
..., "exceptions": ..., "exchange_available": bool}.
    - If the category is not found, return {"error": True, "message": f"No
refund policy for category: {category}."}.
```

```
4. escalate_to_human(reason: str) -> dict
    - Accept a reason string.
    - Generate a fake ticket_id using: "ESC-" + a random 6-digit number.
    - Return: {"escalated": True, "ticket_id": ..., "reason": reason,
"message": "A human agent will follow up within 2 business hours.",
"priority": "high"}.
```

Requirements:

- At the top, load fake_orders.json once into a module-level variable using pathlib to find the file relative to tools.py itself (so it works regardless of where the script is run from).
- Input validation: if order_id or category is not a non-empty string, return {"error": True, "message": "Invalid input."}.
- No function should raise an exception on bad user input.
- Include docstrings.
- Output only the Python code, no explanation.

Complete File Contents:

```
"""
tools.py - Customer-service agent tools.
These are plain Python functions. Gemini will decide when to call them.
"""
```

```
import json
import random
from pathlib import Path
```

```
# Load the orders database once at module level.
# Path(__file__).parent finds the directory where tools.py lives,
# so this works no matter where you run the script from.
_DATA_PATH = Path(__file__).parent / "fake_orders.json"
```

```
with open(_DATA_PATH, "r", encoding="utf-8") as _f:
    ORDERS_DB: dict = json.load(_f)
```

```
# -----
--
# Tool 1: get_order
# -----
--
```

```
def get_order(order_id: str) -> dict:
    """
    Return full order details for a given order_id.
```

```

    Args:
        order_id: The order ID as a string, e.g. "1001".

    Returns:
        Order dict if found, or an error dict if not.
    """
    if not isinstance(order_id, str) or not order_id.strip():
        return {"error": True, "message": "Invalid input: order_id must be
a non-empty string."}

    order = ORDERS_DB.get(order_id.strip())
    if order is None:
        return {"error": True, "message": f"Order {order_id} not found."}
    return order

# -----
--
# Tool 2: get_shipping
# -----
--

def get_shipping(order_id: str) -> dict:
    """
    Return shipping status for a given order_id.

    Args:
        order_id: The order ID as a string.

    Returns:
        Dict with shipping fields, or an error dict if not found.
    """
    if not isinstance(order_id, str) or not order_id.strip():
        return {"error": True, "message": "Invalid input: order_id must be
a non-empty string."}

    order = ORDERS_DB.get(order_id.strip())
    if order is None:
        return {"error": True, "message": f"No shipping info for order
{order_id}."}

    return {
        "order_id": order["order_id"],
        "shipping_status": order["shipping_status"],
        "tracking_number": order.get("tracking_number"),
        "estimated_delivery": order.get("estimated_delivery"),
        "delivered_date": order.get("delivered_date"),
    }

# -----
--
# Tool 3: check_refund_policy

```

```
# -----  
--  
  
def check_refund_policy(category: str) -> dict:  
    """  
    Return the refund policy for a product category.  
  
    Args:  
        category: Product category string, e.g. "electronics".  
  
    Returns:  
        Policy dict, or an error dict if category not recognised.  
    """  
    if not isinstance(category, str) or not category.strip():  
        return {"error": True, "message": "Invalid input: category must be  
a non-empty string."}  
  
    policies = {  
        "electronics": {  
            "category": "electronics",  
            "return_window_days": 30,  
            "conditions": "No refund on opened items unless defective.  
Original packaging required.",  
            "exceptions": "Defective items qualify for full refund or  
replacement regardless of packaging.",  
            "exchange_available": True,  
        },  
        "clothing": {  
            "category": "clothing",  
            "return_window_days": 60,  
            "conditions": "Free returns accepted. Item must be unworn with  
tags attached.",  
            "exceptions": "Sale items are final sale. Swimwear and  
underwear cannot be returned for hygiene reasons.",  
            "exchange_available": True,  
        },  
        "home": {  
            "category": "home",  
            "return_window_days": 30,  
            "conditions": "Item must be unused and in original packaging.",  
            "exceptions": "Large furniture requires special return shipping  
arrangement.",  
            "exchange_available": False,  
        },  
        "beauty": {  
            "category": "beauty",  
            "return_window_days": 14,  
            "conditions": "Unopened products only. Sealed items returned in  
original condition.",  
            "exceptions": "Allergic reactions qualify for full refund with  
medical note, even if opened.",  
            "exchange_available": False,  
        },  
        "books": {
```

```

        "category": "books",
        "return_window_days": 30,
        "conditions": "Physical books accepted if in original
condition. No refund on digital/e-book purchases.",
        "exceptions": "Damaged or incorrect books shipped are eligible
for full refund.",
        "exchange_available": False,
    },
    "toys": {
        "category": "toys",
        "return_window_days": 30,
        "conditions": "Must be in original packaging, unopened.
Defective toys accepted opened.",
        "exceptions": "Missing parts or manufacturer defects qualify
for replacement.",
        "exchange_available": True,
    },
    "groceries": {
        "category": "groceries",
        "return_window_days": 7,
        "conditions": "Non-perishable items only. Perishables are non-
refundable once delivered.",
        "exceptions": "Damaged, spoiled, or incorrect items qualify for
full refund regardless of perishability.",
        "exchange_available": False,
    },
}

key = category.strip().lower()
policy = policies.get(key)
if policy is None:
    return {"error": True, "message": f"No refund policy found for
category: '{category}'."}
return policy

# -----
--
# Tool 4: escalate_to_human
# -----
--

def escalate_to_human(reason: str) -> dict:
    """
    Escalate a customer issue to a human agent.

    Args:
        reason: A description of why escalation is needed.

    Returns:
        Dict confirming escalation with a ticket ID.
    """
    if not isinstance(reason, str) or not reason.strip():
        return {"error": True, "message": "Invalid input: reason must be a

```

```

non-empty string."}

ticket_id = f"ESC-{{random.randint(100000, 999999)}}"
return {
    "escalated": True,
    "ticket_id": ticket_id,
    "reason": reason.strip(),
    "message": "A human agent will follow up within 2 business hours.",
    "priority": "high",
}

```

agent.py

DeepSeek Prompt:

Write a complete Python file called agent.py that implements a customer-service agent using the Google Gemini API with manual function calling (not automatic).

Requirements:

- Use the google-genai SDK (pip install google-genai).
- Load GEMINI_API_KEY from environment variables (use python-dotenv to load .env).
- Load GEMINI_MODEL from environment, defaulting to "gemini-1.5-flash" if not set.
- Import and use 4 tool functions from tools.py: get_order, get_shipping, check_refund_policy, escalate_to_human.
- Define Gemini function declarations (tool schemas) for all 4 tools using the google.genai.types.Tool and FunctionDeclaration classes.
- Implement a run_agent(user_message: str) function that:
 1. Creates a chat with Gemini including the tool definitions.
 2. Sends the user message.
 3. Checks if Gemini returned a function_call.
 4. If yes: extracts the function name and arguments, calls the matching local Python function, sends the result back as a function_response.
 5. Loops until Gemini returns a final text response (no more function calls).
 6. Has a max loop limit of 6 to prevent infinite loops.
 7. Prints each tool call clearly: ">>> Tool called: <name> with args: <args>".
 8. Returns the final text response.
- Implement a main() that supports two modes via command-line argument:
 - "chat" mode: interactive loop where the user types queries.
 - "test" mode: runs 10 predefined test queries from a list and prints results.
- Include a SYSTEM_PROMPT string that tells Gemini it is a customer-service agent for ShopEase, should use tools to look up real data, and should escalate frustrated customers.
- Handle errors: unknown tool name, malformed arguments (catch exceptions), API errors, max loop exceeded.

- Include clear comments throughout for interns.
- Output only the Python code, no explanation.

Complete File Contents:

```
"""
agent.py - Gemini function-calling customer-service agent for ShopEase.
```

```
Run modes:
```

```
python agent.py chat    -> interactive CLI chat
python agent.py test    -> run 10 predefined test queries
```

```
"""
```

```
import os
import sys
import json
from dotenv import load_dotenv
import google.generativeai as genai
```

```
# Import our 4 local tool functions
from tools import get_order, get_shipping, check_refund_policy,
escalate_to_human
```

```
# -----
--
# Configuration
# -----
--
```

```
load_dotenv() # Load .env file if present
```

```
API_KEY = os.getenv("GEMINI_API_KEY")
if not API_KEY:
    print("ERROR: GEMINI_API_KEY environment variable is not set.")
    print("Create a .env file with: GEMINI_API_KEY=your_key_here")
    sys.exit(1)
```

```
MODEL_NAME = os.getenv("GEMINI_MODEL", "gemini-1.5-flash")
MAX_TOOL_CALLS = 6 # Safety limit to prevent infinite loops
```

```
genai.configure(api_key=API_KEY)
```

```
# -----
--
# System prompt - tells Gemini who it is and how to behave
# -----
--
```

```
SYSTEM_PROMPT = """
You are a helpful customer-service agent for ShopEase, an online retail
store.
```


Your job is to help customers with:

- Order status and details
- Shipping and delivery information
- Refund and return policies
- Escalating complex issues to human agents

Rules:

1. Always use the available tools to look up real data. Do not guess.
2. If a customer seems frustrated, angry, or has a disputed order with a denied refund,
 - use `escalate_to_human` with a clear reason summarising the situation.
3. Be polite, concise, and helpful.
4. If a tool returns an error, tell the customer clearly and offer to escalate.
5. When checking a refund policy, use the product category from the order details.

"""

```
# -----
--
# Tool definitions (function declarations for Gemini)
# These schemas tell Gemini what each tool is called, what it does,
# and what arguments it expects.
# -----
--
```

```
TOOLS = [
    genai.protos.Tool(
        function_declarations=[
            genai.protos.FunctionDeclaration(
                name="get_order",
                description="Retrieve full order details for a given order
ID, including item, price, status, and refund information.",
                parameters=genai.protos.Schema(
                    type=genai.protos.Type.OBJECT,
                    properties={
                        "order_id": genai.protos.Schema(
                            type=genai.protos.Type.STRING,
                            description="The order ID as a string, e.g.
'1001'."
                        )
                    },
                    required=["order_id"]
                )
            ),
            genai.protos.FunctionDeclaration(
                name="get_shipping",
                description="Get the shipping and delivery status for a
given order ID, including tracking number and estimated delivery date.",
                parameters=genai.protos.Schema(
                    type=genai.protos.Type.OBJECT,
                    properties={
                        "order_id": genai.protos.Schema(
                            type=genai.protos.Type.STRING,
```

```

        description="The order ID as a string."
    )
    },
    required=["order_id"]
)
),
genai.protos.FunctionDeclaration(
    name="check_refund_policy",
    description="Return the refund and return policy for a
product category such as electronics, clothing, home, beauty, books, toys,
or groceries.",
    parameters=genai.protos.Schema(
        type=genai.protos.Type.OBJECT,
        properties={
            "category": genai.protos.Schema(
                type=genai.protos.Type.STRING,
                description="Product category string, e.g.
'electronics'."
            )
        },
        required=["category"]
    )
),
genai.protos.FunctionDeclaration(
    name="escalate_to_human",
    description="Escalate the customer's issue to a human
agent. Use this when the customer is frustrated, has a complex dispute, or
when standard tools cannot resolve the issue.",
    parameters=genai.protos.Schema(
        type=genai.protos.Type.OBJECT,
        properties={
            "reason": genai.protos.Schema(
                type=genai.protos.Type.STRING,
                description="A clear explanation of why this is
being escalated, including relevant order IDs and issue details."
            )
        },
        required=["reason"]
    )
),
]

# -----
--
# Map tool names to local Python functions
# -----
--

TOOL_REGISTRY = {
    "get_order": get_order,
    "get_shipping": get_shipping,
    "check_refund_policy": check_refund_policy,

```

```

    "escalate_to_human": escalate_to_human,
}

# -----
--
# Core agent function
# -----
--

def run_agent(user_message: str) -> str:
    """
    Send a user message to Gemini, handle tool calls in a loop,
    and return the final natural-language response.

    The loop:
    1. Send message -> Gemini may return a function_call
    2. Execute the function locally
    3. Send the result back as function_response
    4. Repeat until Gemini returns a text response
    5. Stop after MAX_TOOL_CALLS iterations for safety
    """

    # Initialise the model and start a chat session
    model = genai.GenerativeModel(
        model_name=MODEL_NAME,
        system_instruction=SYSTEM_PROMPT,
        tools=TOOLS,
    )

    # The conversation history (grows as we add messages and responses)
    chat = model.start_chat(history=[])

    # Track number of tool calls to prevent infinite loops
    tool_call_count = 0

    # The current message to send (starts as the user's message)
    current_message = user_message

    print(f"\n{'='*60}")
    print(f"User: {user_message}")
    print(f"{'='*60}")

    while True:
        # --- Step 1: Send the current message to Gemini ---
        try:
            response = chat.send_message(current_message)
        except Exception as e:
            return f"[API Error] Failed to get response from Gemini: {e}"

        # --- Step 2: Check what Gemini returned ---
        # Gemini can return either:
        # (a) A function_call - it wants us to run a tool
        # (b) Text - it's giving a final answer

```

```

candidate = response.candidates[0]
parts = candidate.content.parts

# Check if any part is a function call
function_call_part = None
for part in parts:
    if hasattr(part, "function_call") and part.function_call.name:
        function_call_part = part.function_call
        break

if function_call_part is None:
    # --- No function call: Gemini gave a final text answer ---
    final_text = response.text
    print(f"\nAgent: {final_text}")
    return final_text

# --- Step 3: We have a function call ---
tool_call_count += 1

if tool_call_count > MAX_TOOL_CALLS:
    # Safety: stop if we've looped too many times
    warning = "[Safety limit reached] Too many tool calls.
Stopping."
    print(warning)
    return warning

tool_name = function_call_part.name
# Arguments come as a Struct – convert to a plain Python dict
try:
    tool_args = dict(function_call_part.args)
except Exception:
    tool_args = {}

print(f"\n>>> Tool called: {tool_name} with args: {tool_args}")

# --- Step 4: Execute the matching local function ---
if tool_name not in TOOL_REGISTRY:
    # Gemini called a tool that doesn't exist – tell it that
    tool_result = {"error": True, "message": f"Unknown tool:
{tool_name}"}
else:
    tool_func = TOOL_REGISTRY[tool_name]
    try:
        tool_result = tool_func(**tool_args)
    except TypeError as e:
        # Arguments didn't match the function signature
        tool_result = {"error": True, "message": f"Bad arguments
for {tool_name}: {e}"}
    except Exception as e:
        # Any other unexpected error in the tool
        tool_result = {"error": True, "message": f"Tool {tool_name}
failed: {e}"}

print(f"    Result: {json.dumps(tool_result, indent=2)}")

```

```

        # --- Step 5: Send the tool result back to Gemini ---
        # We send it as a function_response so Gemini knows what the tool
returned.
        # Then the loop continues – Gemini will either call another tool or
give a final answer.
        current_message = genai.protos.Content(
            parts=[
                genai.protos.Part(
                    function_response=genai.protos.FunctionResponse(
                        name=tool_name,
                        response={"result": tool_result},
                    )
                )
            ]
        )

# -----
--
# Predefined test queries (for test mode)
# -----
--

TEST_QUERIES = [
    # Single-tool queries
    "What is the status of order 1001?",
    "What's the shipping status of order 1003?",
    "What is the refund policy for electronics?",

    # Multi-tool queries
    "Can I refund order 1007? Has it been delivered?",
    "Tell me everything about order 1005 – what happened and can I get a
refund?",
    "Order 1010 was delivered but isn't working. What are my options?",

    # Multi-turn simulations (single messages that imply multi-step
reasoning)
    "I ordered a beauty product (order 1013) and had an allergic reaction.
What's the policy and what should I do?",
    "My order 1012 is late. Can you check the shipping and tell me the
clothing return policy just in case?",

    # Escalation triggers
    "This is absolutely ridiculous. Order 1016 arrived damaged, you denied
my refund, and nobody has helped me. I want to speak to a manager NOW.",
    "I've been waiting for weeks. Order 1002 is still in transit, your
tracking is useless, and I want a refund immediately. I'm furious.",
]

# -----
--
# Entry point

```

```
# -----
--

def main():
    mode = sys.argv[1] if len(sys.argv) > 1 else "chat"

    if mode == "test":
        print(f"\nRunning {len(TEST_QUERIES)} test queries against
{MODEL_NAME}...\n")
        for i, query in enumerate(TEST_QUERIES, 1):
            print(f"\n--- Test {i}/{len(TEST_QUERIES)} ---")
            run_agent(query)
        print("\n\nAll tests complete.")

    elif mode == "chat":
        print(f"\nShopEase Customer-Service Agent ({MODEL_NAME})")
        print("Type 'quit' or 'exit' to stop.\n")
        while True:
            try:
                user_input = input("You: ").strip()
            except (EOFError, KeyboardInterrupt):
                print("\nGoodbye!")
                break
            if not user_input:
                continue
            if user_input.lower() in ("quit", "exit"):
                print("Goodbye!")
                break
            run_agent(user_input)

    else:
        print(f"Unknown mode: '{mode}'. Use 'chat' or 'test'.")
        sys.exit(1)

if __name__ == "__main__":
    main()
```

requirements.txt

DeepSeek Prompt:

Write a requirements.txt file for a Python project that uses:

- google-generativeai (the google-genai SDK for Gemini)
- python-dotenv (for loading .env files)

Pin to reasonable stable versions that would have been available in mid-2024.

Output only the requirements.txt contents, no explanation.

Complete File Contents:

```
google-generativeai>=0.7.0
python-dotenv>=1.0.0
```

.env.example**DeepSeek Prompt:**

```
Write a .env.example file for a Python Gemini project.
Include GEMINI_API_KEY (empty value with a comment explaining where to get
it)
and GEMINI_MODEL (defaulting to gemini-1.5-flash with a comment).
Output only the file contents.
```

Complete File Contents:

```
# Copy this file to .env and fill in your values.
# Never commit .env to version control.

# Get your API key from: https://aistudio.google.com/app/apikey
GEMINI_API_KEY=

# Optional: override the model. Defaults to gemini-1.5-flash if not set.
GEMINI_MODEL=gemini-1.5-flash
```

test_queries.md**DeepSeek Prompt:**

```
Write a test_queries.md file for a customer-service agent project using
Gemini function calling.

Include exactly 10 test queries in 4 categories:
- 3 single-tool queries (each uses only 1 of: get_order, get_shipping,
check_refund_policy, escalate_to_human)
- 3 multi-tool queries (require 2 or more tool calls)
- 2 multi-turn examples (simulate a conversation where the customer adds
more context)
- 2 escalation triggers (angry/frustrated customer messages that should
trigger escalate_to_human)

For each query, include:
- Test ID (T01-T10)
```

- Category
- The exact query text
- Expected tools that should be called
- What a good response looks like

Use order IDs from 1001–1022. Be specific and realistic.
Output only the Markdown, no explanation.

Complete File Contents:

Test Queries – ShopEase Customer-Service Agent

Run these with: ``python agent.py test``
Or paste them one by one in: ``python agent.py chat``

Category 1: Single-Tool Queries

T01 – get_order

****Query:**** *What is the status of order 1001?*

****Expected tools:**** ``get_order``

****Good response:**** Should state the order is delivered, mention the item (Wireless Noise-Cancelling Headphones), the price (\$149.99), and that it arrived on 2024-05-09.

T02 – get_shipping

****Query:**** *What's the shipping status of order 1003?*

****Expected tools:**** ``get_shipping``

****Good response:**** Should confirm the Air Fryer was delivered on 2024-05-21 with tracking number TRK-CN1003.

T03 – check_refund_policy

****Query:**** *What is the refund policy for electronics?*

****Expected tools:**** ``check_refund_policy``

****Good response:**** Should explain the 30-day window, that opened items are not refundable unless defective, and that exchanges are available.

Category 2: Multi-Tool Queries

T04 – get_order + check_refund_policy

****Query:**** *Can I refund order 1007? Has it been delivered?*

****Expected tools:**** ``get_order``, ``check_refund_policy``

****Good response:**** Should confirm the Women's Yoga Pants were delivered, then explain the 60-day clothing return window and free returns policy.

T05 – get_order + check_refund_policy

****Query:**** Tell me everything about order 1005 – what happened and can I get a refund?

****Expected tools:**** `get\_order`, `check\_refund\_policy`

****Good response:**** Should describe the disputed Vitamin C Serum delivery, the pending refund, and explain that the beauty policy allows refunds for damaged items. May suggest escalation.

T06 – get_order + get_shipping + check_refund_policy

****Query:**** Order 1010 was delivered but the camera isn't working. What are my options?

****Expected tools:**** `get\_order`, `get\_shipping`, `check\_refund\_policy`

****Good response:**** Should confirm delivery, note the dispute and pending refund, explain electronics policy (defective items qualify for refund), and likely recommend escalation given the notes.

Category 3: Multi-Turn Conversation Examples

T07 – get_order + check_refund_policy (context-heavy)

****Query (turn 1):**** I ordered a beauty product and had an allergic reaction. The order ID is 1013. What's the return policy?

****Expected tools:**** `get\_order`, `check\_refund\_policy`

****Good response:**** Should find the Retinol Night Cream (already returned), note the refund is approved, and confirm that allergic reactions are an exception to the 14-day sealed-only policy.

T08 – get_shipping + check_refund_policy

****Query:**** My order 1012 is late. Can you check the shipping and also tell me the clothing return policy just in case?

****Expected tools:**** `get\_shipping`, `check\_refund\_policy`

****Good response:**** Should report in-transit status, note the 3-day delay, provide the tracking number, and explain the 60-day clothing return window.

Category 4: Escalation Triggers

T09 – get_order + escalate_to_human

****Query:**** This is absolutely ridiculous. Order 1016 arrived damaged, you denied my refund, and nobody has helped me. I want to speak to a manager NOW.

****Expected tools:**** `get\_order`, `escalate\_to\_human`

****Good response:**** Should retrieve the order details (mechanical keyboard, denied refund), acknowledge the frustration, and escalate with a ticket ID and estimated follow-up time.

```
---

### T10 – get_shipping + escalate_to_human
**Query:** I've been waiting for weeks. Order 1002 is still in transit,
your tracking is useless, and I want a refund immediately. I'm furious.
**Expected tools:** `get_shipping`, `escalate_to_human`
**Good response:** Should retrieve shipping status, acknowledge the delay,
and escalate with a clear reason including the order ID and customer
sentiment.
```

test_results.md

DeepSeek Prompt:

```
Write a test_results.md file for a Gemini function-calling customer-service
agent project.

Include:
1. A results table with these columns: Test ID, Query (short), Expected
Tools, Actual Tools Called, Final Response Summary, Pass/Fail, Notes.
2. Rows T01 through T10.
3. For T01–T03, fill in sample rows showing what a PASS looks like
(realistic expected tool, realistic response summary, PASS, short note).
4. For T04–T10, leave Actual Tools Called, Final Response Summary, and
Pass/Fail as blank placeholders (e.g. "-") for the intern to fill in after
running the tests.
5. Add a section at the bottom with evaluation criteria: what counts as
PASS vs FAIL.

Output only the Markdown, no explanation.
```

Complete File Contents:

```
# Test Results – ShopEase Customer-Service Agent

Fill in the "Actual Tools Called", "Final Response Summary", and
"Pass/Fail" columns after running each test.

Run all tests: `python agent.py test`

---

## Results Table

| Test ID | Query (short) | Expected Tools | Actual Tools Called | Final
Response Summary | Pass/Fail | Notes |
|-----|-----|-----|-----|-----|
-----|-----|-----|
```

```

| T01 | Status of order 1001? | `get_order` | `get_order` | Confirmed
delivered 2024-05-09, Headphones $149.99, no refund requested | ✅ PASS |
Clean single-tool call |
| T02 | Shipping status of 1003? | `get_shipping` | `get_shipping` |
Delivered 2024-05-21, tracking TRK-CN1003 | ✅ PASS | Correct tool,
correct data |
| T03 | Refund policy for electronics? | `check_refund_policy` |
`check_refund_policy` | 30-day window, no refund on opened items unless
defective, exchanges available | ✅ PASS | Policy returned accurately |
| T04 | Can I refund order 1007? | `get_order`, `check_refund_policy` | - |
- | - | |
| T05 | Everything about order 1005? | `get_order`, `check_refund_policy` |
- | - | - | |
| T06 | Order 1010 camera not working? | `get_order`, `get_shipping`,
`check_refund_policy` | - | - | - | |
| T07 | Allergic reaction, order 1013? | `get_order`, `check_refund_policy`
| - | - | - | |
| T08 | Order 1012 late, check shipping + policy? | `get_shipping`,
`check_refund_policy` | - | - | - | |
| T09 | Furious about denied refund, order 1016 | `get_order`,
`escalate_to_human` | - | - | - | |
| T10 | Furious, order 1002 stuck in transit | `get_shipping`,
`escalate_to_human` | - | - | - | |

```

Evaluation Criteria

✅ PASS if ALL of the following are true:

- The correct tools were called (extra tool calls are acceptable if logical).
- Arguments passed to tools were correct (right *order_id*, right *category*).
- The final response accurately reflects what the tool returned – no hallucinated data.
- For escalation tests: `escalate_to_human` was called with a meaningful reason.
- The response tone is appropriate (helpful, not dismissive).

❌ FAIL if ANY of the following:

- Wrong tool called (e.g. `get_order` when `get_shipping` was needed).
- Correct tool called but with wrong argument (e.g. wrong *order_id*).
- Final answer contradicts the tool result (Gemini ignored or hallucinated over the data).
- Escalation not triggered when it clearly should be.
- Infinite loop hit (MAX_TOOL_CALLS exceeded).
- API error not handled gracefully.

⚠️ PARTIAL:

- Correct tools called but response included unnecessary caveats or wrong details.
- Escalation triggered but reason was vague or missing key context.

escalation_logic.md

DeepSeek Prompt:

Write an escalation_logic.md file for a customer-service agent project.

Explain in clear Markdown:

1. What escalation means in this agent context.
2. The 5 main triggers that should cause `escalate_to_human` to be called:
 - Angry or frustrated customer language
 - Disputed orders with denied refunds
 - Missing order after repeated lookup attempt
 - Refund exception cases (allergic reaction, defective item where standard policy doesn't apply)
 - Complex multi-issue situations beyond what the tools can resolve
3. For each trigger, give 1-2 example customer messages that would qualify.
4. What information should be included in the reason argument passed to `escalate_to_human`.
5. A decision flowchart (use ASCII or a simple numbered decision tree, not Mermaid).
6. What the agent should say to the customer before and after escalating.

Write for Gen AI interns. Use headings and bullet points.
Output only the Markdown.

Complete File Contents:

```
# Escalation Logic – ShopEase Customer-Service Agent
```

```
## What Is Escalation?
```

Escalation means handing the customer off to a real human agent when the automated tools can't resolve the situation. In this agent, escalation is done by calling `escalate_to_human(reason)`, which creates a support ticket and promises a human follow-up within 2 business hours.

****Escalation is not failure.**** It's the right call when the tools have given you all the information they can, but the situation still needs human judgment, authority, or empathy.

```
## Trigger 1: Angry or Frustrated Customer Language
```

****When:**** The customer uses language signalling high distress, anger, threats to leave, or demands for management.

****Signal words and phrases:****

- "This is ridiculous / unacceptable"
- "I want to speak to a manager"

- "I'm furious / absolutely fed up"
- "I'll never shop here again"
- "Sue you / report you"

****Example messages:****

> "This is absolutely ridiculous. I've been waiting 3 weeks and nobody is helping me. I want a manager NOW."

> "Your service is useless. I'm done. Refund me immediately or I'm disputing with my bank."

****Action:**** Retrieve relevant order info first if possible, then escalate with the customer's sentiment noted in the reason.

Trigger 2: Disputed Order with Denied Refund

****When:**** `get\_order` returns `order\_status: disputed` AND `refund\_status: denied`.

****Why:**** A denied refund on a disputed order is a high-conflict situation. The automated tools cannot override a refund decision – only a human can.

****Example messages:****

> "Order 1016 arrived broken and you denied my refund. What is going on?"

> "I disputed my order weeks ago and got denied. I need this fixed."

****Action:**** Call `get\_order` to confirm the dispute and denial, then escalate with the order ID and denial context in the reason.

Trigger 3: Missing Order After Repeated Lookup

****When:**** `get\_order` or `get\_shipping` returns an error (order not found), and the customer insists the order exists.

****Why:**** The fake database is finite. If a customer has an order ID that isn't in the system, it may be a data issue, fraud, or a different platform – all requiring human investigation.

****Example messages:****

> "My order ID is 1099. I have the confirmation email but you're telling me it doesn't exist?"

> "I placed an order last Monday. Why can't you find it?"

****Action:**** Attempt lookup once, confirm the error, then escalate with a note that the order ID couldn't be found and the customer should provide their confirmation email to the human agent.

Trigger 4: Refund Exception Cases

****When:**** The customer's situation qualifies for a policy exception (e.g. allergic reaction, defective product, incorrect item shipped) but the standard policy would normally deny them.

****Why:**** The `check\_refund\_policy` tool returns policy text, but it cannot authorise an exception. A human must approve it.

****Example messages:****

> "I'm allergic to the cream you sent. I opened it and had a reaction. I know beauty items are final sale but this is a health issue."

> "The puzzle I ordered arrived with 50 missing pieces. You're saying I need original packaging for a return but the packaging is the one you sent me."

****Action:**** Call `check\_refund\_policy` to identify the relevant exception clause, then escalate with the specific exception context so the human agent knows what to approve.

Trigger 5: Complex Multi-Issue Situations

****When:**** The customer has multiple overlapping problems that exceed what 6 tool calls can resolve, or the situation involves a combination of the above.

****Example messages:****

> "Order 1010 – I have a dispute open, the refund is pending, the camera doesn't work, and I've called three times already."

> "This is my second order this month with an issue. My previous order was 1004 and now 1016 is broken too."

****Action:**** Gather what you can from the tools, then escalate with a summary of all issues.

What to Include in the `reason` Argument

The `reason` string passed to `escalate\_to\_human` should always include:

- ****Order ID(s)**** involved
- ****The issue**** in plain language
- ****Current order/refund status**** from the tools
- ****Customer sentiment**** (e.g. "Customer is very frustrated")
- ****What was already tried**** (e.g. "Refund policy checked – exception may apply")

****Good reason example:****

Order 1016 (Mechanical Keyboard, \$109). Order status: disputed. Refund status: denied. Customer is very frustrated and reporting missing keycaps. Standard policy states no defect found, but customer disputes this. Human review required for refund decision override.

****Bad reason example:****

Customer is angry.

Escalation Decision Tree

Customer message received | v Does the customer use angry/frustrated language? YES → Retrieve order info if available → ESCALATE NO ↓ Does the order have status=disputed AND refund_status=denied? YES → ESCALATE NO ↓ Is the order not found after lookup? YES → ESCALATE NO ↓ Does the situation require a policy exception? YES → Check policy, identify exception → ESCALATE NO ↓ Does the issue involve multiple unresolved problems? YES → Gather context → ESCALATE NO ↓ Resolve with tools and give final answer.

What to Say to the Customer

Before escalating:

> "I can see this situation needs personal attention. Let me connect you with one of our specialist agents who can resolve this directly."

After escalating (include the ticket ID from the tool result):

> "I've escalated your case to our team. Your ticket ID is ****ESC-XXXXXX****. A human agent will follow up within 2 business hours. I'm sorry for the inconvenience."

Key Principle

****Escalate early on emotional signals.**** It is much better to escalate a

customer who didn't strictly need it than to frustrate a customer who did by refusing to escalate. When in doubt, escalate.

learnings.md

DeepSeek Prompt:

Write a learnings.md file for a Gen AI intern who has just built a Gemini function-calling agent.

The file should be a reflection and debugging journal template with these sections:

1. What I tried to do
2. What broke (error message or unexpected behaviour)
3. What Gemini tried to do (which tool did it call, or did it answer directly?)
4. Which tool was expected vs actually called
5. Were the arguments correct? (What did Gemini pass in?)
6. Was the tool result useful? (Did the function return the right data?)
7. Did the final answer match the tool result? (Or did Gemini hallucinate/ignore it?)
8. How I fixed it
9. What I learned about function calling

Also include a "Key Lessons" section at the end with 5 pre-written insights about function calling that the intern should internalise after completing Day 3.

Write in a friendly, direct tone for a Day 3 intern. Leave the main sections as fill-in templates with guiding questions. Output only the Markdown.

Complete File Contents:

Learnings Journal – Day 3: Gemini Function Calling Agent

Use this file to record what broke, how you fixed it, and what you learned. Fill in one entry each time something goes wrong or surprises you. This is the most valuable document you'll write today.

Entry Template (copy and fill in for each issue)

Entry #___ – Date: _____

1. What I tried to do

> What query did you send? What did you expect to happen?

Write here...

2. What broke

- > Describe the error message, unexpected output, or wrong behaviour.
- > Copy the exact error if there is one.

Write here...

3. What Gemini tried to do

- > Did Gemini call a tool? Which one? Did it skip tools and answer directly?
- > Did it call the wrong tool? Did it loop?

Write here...

4. Expected tool vs actual tool called

Tool Name	Arguments passed
Expected	
Actual	

5. Were the arguments correct?

- > What exactly did Gemini pass as arguments?
- > Were the types correct (e.g. string vs int)?
- > Was the *order_id* or *category* spelled correctly?

Write here...

6. Was the tool result useful?

- > What did the Python function actually return?
- > Did it match what you expected it to return?
- > Was there an error dict?

Write here...

7. Did the final answer match the tool result?

- > Did Gemini accurately reflect what the tool returned?
- > Did it miss any key information?
- > Did it add anything that wasn't in the tool result (hallucination)?

Write here...

8. How I fixed it

> What change did you make?

> Was it in tools.py, agent.py, the schema, or the system prompt?

Write here...

9. What I learned about function calling

> One sentence that captures the lesson.

Write here...

Key Lessons (Read These After You Finish)

Once you've tested all 10 queries and debugged at least one issue, read these:

Lesson 1: Gemini doesn't run your code – you do

Gemini tells you **what function to call and with what arguments**. Your Python code runs it and sends the result back. If you don't send the result back correctly, Gemini is blind to what happened. Most bugs come from the send-back step being wrong.

Lesson 2: The function schema is a contract

If your schema says ``order_id`` is a string and your function expects a string, you're fine. If there's a mismatch – even just in the description – Gemini may pass the wrong type or the wrong field name. Keep your schemas and your function signatures in sync.

Lesson 3: Gemini reads the tool result, not your code

Gemini has no idea what ``get_order`` does internally. It only sees the JSON you send back as ``function_response``. If your tool returns ``{"error": True, "message": "..."}``, Gemini will read that and (hopefully) tell the customer clearly. This is why structured error returns matter more than raising exceptions.

Lesson 4: The system prompt shapes tool-use decisions

Gemini's decision of **when** to call a tool (and which one) is heavily influenced by the system prompt. If Gemini keeps answering without calling tools, your system prompt probably needs to be more explicit: "Always use `get_order` to look up real data. Do not guess."

Lesson 5: Escalation is a tool like any other

``escalate_to_human`` is just another function that returns a dict. The intelligence is in Gemini deciding **when** to call it based on customer sentiment and context. You control that decision by writing good

descriptions in the function declaration and clear instructions in the system prompt.

Step 3: Install Dependencies and Run

Install Python packages

```
cd fc_agent
pip install -r requirements.txt
```

Set up your API key

```
# macOS/Linux
cp .env.example .env
nano .env # or open in any editor

# Windows PowerShell
Copy-Item .env.example .env
notepad .env
```

Add your Gemini API key (get one free at <https://aistudio.google.com/app/apikey>):

```
GEMINI_API_KEY=your_actual_key_here
```

Run the interactive chat agent

```
python agent.py chat
```

You'll see a prompt. Type any customer query and press Enter.

Run all 10 test queries automatically

```
python agent.py test
```

This runs through all 10 predefined queries and prints every tool call + final response.

Inspect tool-call traces

Every tool call prints to the terminal in this format:

```
>>> Tool called: get_order with args: {'order_id': '1003'}
Result: {
  "order_id": "1003",
  "customer_name": "Carla Nguyen",
  ...
}
```

Read these traces carefully. They show you exactly what Gemini decided to call and what your Python code returned. This is your debugging window.

Step 4: How to Fill In Your Results

After running each test, open `test_results.md` and fill in:

- **Actual Tools Called** — copy from the `>>> Tool called:` lines in the terminal.
 - **Final Response Summary** — 1–2 sentences summarising what the agent told the "customer".
 - **Pass/Fail** — compare against the evaluation criteria in `test_results.md`.
 - **Notes** — anything unexpected: wrong args, hallucinated data, missed escalation.
-

Troubleshooting

Missing API key

```
ERROR: GEMINI_API_KEY environment variable is not set.
```

Fix: Make sure you created `.env` from `.env.example` and added your key. The `.env` file must be in the `fc_agent/` folder (same level as `agent.py`).

Wrong package installed

```
ModuleNotFoundError: No module named 'google.generativeai'
```

Fix:

```
pip install google-generativeai
```

Make sure you didn't accidentally install `google-genai` (different package). The import in `agent.py` is `import google.generativeai as genai`.

Gemini does not call tools at all

Symptom: Gemini answers your query without triggering any `>>> Tool called:` lines.

Causes and fixes:

- The tool descriptions in the schema are too vague — make them more specific.
 - The system prompt doesn't instruct Gemini strongly enough to use tools. Add: "You must use the provided tools to look up real data. Never answer from memory."
 - The query doesn't match any tool description closely enough. Try rephrasing: "What's the status of order 1001?" instead of "Tell me about 1001".
-

Gemini calls the wrong tool

Symptom: You asked about shipping but Gemini called `get_order`.

Fix: Improve the tool descriptions. `get_shipping` should be described as returning *shipping and delivery status specifically*. `get_order` covers the full order. If both are called, that's often fine — it's being thorough.

JSON file path errors

```
FileNotFoundError: fake_orders.json
```

Fix: The `tools.py` file uses `Path(__file__).parent / "fake_orders.json"` which finds the file relative to `tools.py` itself. Make sure `fake_orders.json` is in the same folder as `tools.py`, not a subfolder.

Tool result not sent back correctly

Symptom: Gemini keeps calling the same tool repeatedly, or gives a final answer that ignores the tool result.

Fix: Check the `current_message` construction in the agent loop. The `function_response` must use exactly the same `name` that Gemini used in its `function_call`. If the names don't match, Gemini treats it as an error and may loop or ignore it.

Infinite loop / repeated tool calls

Symptom: The agent keeps printing `>>> Tool called:` lines until it hits the safety limit.

Fix: The `MAX_TOOL_CALLS = 6` limit will stop it. But to prevent it happening:

- Make sure you're sending `function_response` correctly (see above).
- Make sure the tool returns a useful, non-empty result. Gemini may re-call a tool if it thinks the result was unhelpful.
- Check that you're passing the `Content` object (not a plain string) as `current_message` after a tool call.

Invalid order ID

Symptom: Agent responds that the order doesn't exist, but you're sure you typed 1003.

Fix: Check that `fake_orders.json` contains that order ID as a string key (e.g. `"1003"`, not integer `1003`). The `get_order` function does `ORDERS_DB.get(order_id.strip())`, so there can't be any whitespace. Also check that your JSON file is valid — an unclosed bracket will silently cause all lookups to fail.

Function schema mismatch

Symptom: Gemini passes `orderId` but your function expects `order_id`, causing a `TypeError`.

Fix: The parameter names in your schema (`"order_id"`) must exactly match the Python function argument names. The schema defines what Gemini sends; your function defines what it accepts. If they differ, you'll get a bad-argument error caught by the `TypeError` handler in the loop.

Quick Reference: The Function-Calling Loop

```
[You]                                [Gemini]
|                                    |
|-- user message + tool schemas--> |
|<-- function_call(name, args) --|   |-- thinks...
| run tool locally                  |
|<-- function_response(result) --> |
|<-- [another function_call] ----|   |-- thinks...
| (loop repeats or ends)          |   (or final text)
```

Every arrow is a message you send or receive. The loop is yours to control — Gemini just requests; you decide to execute.

What's Next (Day 4 Preview)

Once this is working, you'll be ready to:

- Add **memory** so the agent remembers earlier messages in a conversation.
- Add **streaming** so responses appear word by word instead of all at once.
- Use **automatic function calling** and compare it to the manual loop you built today.
- Connect real APIs instead of a fake JSON database.

The manual loop you built today is the foundation of every production agent. Don't rush past it.

